Шесть статей про RAG и что из них взять в TenderAI

Что пригодится прямо сейчас, а что подождёт, пока мы не дорастём до агента

3 ноября 2026

1 Агентный RAG на живом примере

Источник: «Пример реализации агентного RAG'а», slivka_83, Хабр, 16 ноября 2025.

Главная мысль статьи

Автор начинает с обычного RAG, и это обычный конвейер. Документ режут на чанки, чанки кладут в векторную базу, по запросу находят ближайшие и отдают модели, а она пишет ответ. Можно добавить реранкер и гибридный поиск, но порядок шагов от этого не меняется: поиск, реранкер, модель.

На простой вопрос вроде «какая прибыль была у Магнита в 2020 году» этого хватает. А вот задачу «как менялась формулировка рисков за три года» так не решить. Её нужно разложить на шаги: узнать текущий год, поискать каждый отчёт отдельно и только потом сравнить формулировки. Жёсткий конвейер сам до этого не додумается.

Отсюда и агентный RAG. Это не какой-то особый ретривер, а отказ от заранее заданного порядка. У модели есть набор инструментов, и поиск по чанкам — лишь один из них. Модель сама решает, что вызвать, может повторить вызов, если результат её не устроил, и копит историю всех вызовов. Ответ она пишет в самом конце, опираясь на эту историю.

Стек у автора игрушечный, и копировать его не обязательно:

- Qdrant со скалярным произведением и размерностью 1536.
- Marker для разбора PDF. Автор сам признаёт, что для RAG могут подойти olmOCR или Docling, потому что Marker вставляет в текст служебные тире.
- Чанки с метаданными, а не голый текст.
- Два инструмента, которые модель видит через MCP: поиск по чанкам и служебный вызов вроде текущей даты.
- Qwen3 14B в роли «мозга», потому что она умеет вызывать функции, и фреймворк Agno в качестве обвязки.

В конце автор сам перечисляет, чего не хватает его примеру: гибридного поиска BM25 вместе с семантикой, фильтров по метаданным и подбора параметров вызова модели. И честно замечает, что список инструментов никогда не бывает закрытым. Для вопроса «у каких трёх компаний самый большой доход» нужна уже таблица, а не векторный поиск.

Минусы, которые он называет, мы видим и в собственном журнале токенов. Служебных следов становится больше, и мониторить систему тяжелее. Токенов уходит больше, причём не каждый вызов окупается. Ответ приходит дольше, а промежуточные вызовы инструментов плохо показывать пользователю потоком.

Что это значит для нашего задания

Наше задание — не «построить RAG». Нужно воспроизводимо сопоставить десять закупок с четырьмя SKU: найти соответствующую характеристику, проверить числа кодом, приложить цитаты и записать расход токенов. GPT-Luna здесь только извлекает требования из прозы, статус она не ставит.

И конвейер для этого у нас уже есть. Сначала идёт инвентаризация файлов, потом извлечение требований, выравнивание характеристики, кодовая проверка и агрегация, и всё это в одном и том же порядке. Модель следующий шаг не выбирает. На прозе GPT-Luna один раз достаёт требования с цитатами, а числа, единицы, множества и отрицания считает код. Это ближе к «ретривер плюс детерминированный вердикт», чем к агенту из статьи.

Поэтому переносить статью в текущий прогон не нужно. Все четыре SKU мы и так сравниваем полностью, и агент, который «решает, искать или нет», только сожжёт токены. К тому же задание прямо запрещает приравнивать близость к соответствию, так что ставить `match` агенту нельзя.

Но кое-что из статьи всё-таки полезно держать в голове уже сейчас.

- Инструмент и ответ модели — разные вещи. Наш `check_constraints` — это инструмент, а GPT-Luna вердикт не выносит.
- Метаданные чанка — файл, страница, позиция, артикул — нужны ещё до того, как появится векторная база. У нас они уже лежат в `Span`.
- Автора не устраивает поиск только по смыслу. Значит, наш лексический канон — не временный костыль, а вторая опора поиска.

А если вырастем до агента

Если продукт дорастёт до вопросов на обычном языке по пачке закупок, каркас из статьи подойдёт. Не подойдёт его начинка — «найди чанк и перескажи».

Инструментами тогда должны стать наши этапы, а не общий поиск по PDF:

1. список файлов закупки и их настоящие типы;
2. извлечение требований: таблицы и XML ЕИС разбирает код, остальное — Luna;
3. выравнивание требования на характеристики одного SKU;
4. кодовая проверка ограничения;
5. подъём статуса на позицию и на закупку.

Агент вправе выбрать порядок и повторить поиск, если цитата оказалась пустой. Но подменить четвёртый пункт текстом модели он не вправе. А историю вызовов, о которой пишет автор, мы уже ведём: это `usage.jsonl` с этапом, моделью, токенами и ошибками.

Qdrant, MCP и Agno — способ упаковки, а не требование к архитектуре. Ставить их стоит тогда, когда чанков станет больше, чем помещается в один прогон по четырём карточкам.

2 Девять видов RAG

Источник: «9 подходов, как сделать свой RAG с блекджеком и… », prodata_ai, Хабр, 21 февраля 2026.

Статья обзорная, поэтому возьму из неё только то, что касается нас. По каждому виду — как обстоят дела сейчас и что изменится в агентной версии.

Обычный RAG. Сейчас мы его не строим: наш ответ — статус с цитатой, а не пересказ чанка. В агенте он станет нижним слоем поиска, если появится вопрос «где в ТЗ это написано».

Conversational RAG. Сейчас не нужен: прогон пакетный, диалога нет. В агенте пригодится, только если пользователь будет уточнять закупку в чате.

Corrective и Self-RAG. Сейчас это дороже, чем наш `unknown`: код и так отказывается отвечать, когда значения нет. В агенте имеет смысл как проверка цитаты — есть ли она в файле и подтверждает ли вывод. Это та самая метрика доказательности из ТЗ.

Adaptive RAG. Он у нас уже есть, только вручную: таблицы и XML идут в код, проза — в Luna. В агенте эту ветку может выбирать сама модель, но сначала стоит убедиться, что

простое правило «таблица пустая — значит, Luna» работает не хуже.

Fusion RAG. Подробнее о нём в разделе **??**. Сливать ранги четырёх SKU бессмысленно. В агенте это несколько формулировок одного требования и RRF по характеристикам.

HyDE. Сейчас он опасен: гипотетический ответ модели уведёт поиск от чисел, а ТЗ ровно об этом и предупреждает. В агенте — только для длинных текстовых требований без чисел, но никак не для объёма, скорости или болюса.

Agentic RAG. Сейчас избыточен, это разобрано в разделе **??**. В агенте это целевая оболочка, но вердикт остаётся инструментом.

GraphRAG. Граф «закупка — позиция — требование — характеристика — SKU» у нас уже есть в JSON, отдельная графовая база для этого не нужна. Понадобится она тогда, когда связи между артикулами и документами перестанут помещаться в код.

Единственный практический вывод, который я из неё делаю, такой. Большинство «продвинутых» RAG борются с галлюцинациями тем, что модель ещё раз перечитывает текст. У нас же галлюцинацию числа ловит код. Покупать Self-RAG или HyDE до провала кодового чекера не нужно.

3 FRIDA, русский энкодер от SberDevices

Источник: «Знакомьтесь, FRIDA. Открытая эмбеддинг-модель для русского языка», SberDevices, Хабр, 20 мая 2025.

FRIDA — открытый энкодер для русского языка. На момент выхода статьи он занимал первое место в ruMTEB. Под капотом у него энкодер FRED-T5, а обучение контрастивное: сначала на парах без ручной разметки, потом на меньшем наборе качественных пар. Любопытная деталь: длинные инструкции модель воспринимает просто как набор ключевых слов, поэтому авторы заменили их короткими префиксами. Для поиска это `search_query:` и `search_document:`.

Для нашего задания FRIDA — кандидат на H2, а не обязательная деталь. ТЗ просит показать, что латентное пространство помогает найти именно характеристику, и сравнить это с простым поиском без него. FRIDA здесь выглядит правдоподобно. Тексты закупок русские, а названия характеристик короткие и часто перефразированы — скажем, «номинальный объём резервуара» и «объём заполнения». Префиксы важно не забыть, иначе модель работает не в режиме поиска.

Обучать свою FRIDA не нужно. В статье контрастивное предобучение на 30 млн пар добавило около 1,8 пункта среднего балла ruMTEB, и это при уже сильном энкодере. У нас же размеченных пар характеристик пока меньше, чем нужно даже для честного recall@k, потому что отложенная выборка ещё не закрыта.

Когда появится агент, FRIDA останется ретривером внутри инструмента «найди характеристику», а не моделью, которая отвечает пользователю.

4 Большой гайд по архитектуре RAG

Источник: «Архитектура RAG: полный гайд», Raft, Хабр, 2 февраля 2024.

Если отбросить обзорную часть, в гайде остаётся несколько мыслей, которые нам пригодятся.

- Дообучать всю модель на документах компании дорого и ненадёжно. RAG — или, как у нас, поиск плюс код — обходится дешевле. Это тот же аргумент, что у LoRA против полного дообучения, только ещё жёстче: знания остаются в документах и обновляются без переобу-

чения.

- Качество упирается в то, что именно нашёл ретривер. Грязные PDF, противоречия и аббревиатуры — это прямо про наш датасет.
- Dense-поиск хорошо ловит перефразы, а sparse (BM25) — точные термины, номера и единицы. Их стоит объединять.
- Ранги автор сливает через RRF. Формула та же, что в статье из раздела **??**, с k около 60.
- Реранкер на LLM дорог. Наш эксперимент H6 уже показал, что поверх четырёх SKU ему просто нечего улучшать.
- Оценивать поиск по «большому косинусу» нельзя, важен порядок: релевантный документ наверху списка ценнее, чем тот же документ в хвосте. Для нас это recall@k по подтверждённой характеристике, причём при четырёх SKU recall@4 ничего не доказывает. Об этом прямо сказано и в ТЗ.
- Низкая уверенность модели по логитам — повод честно ответить «не знаю». У нас эту роль играет `unknown`: значения нет, единицы несовместимы или требование не разобрано.

Прямо сейчас из гайда стоит взять только две вещи: выбор между dense и sparse и метрику, которая учитывает порядок. Нарезка «Войны и мира» в Qdrant к матчингу SKU отношения не имеет.

В агентной версии гайд превращается в чертёж инструмента поиска: перефраз запроса, BM25 вместе с эмбеддингом, RRF, отсечение и цитата. Генерация ответа остаётся последним шагом и проверку ограничения не заменяет.

5 Reciprocal Rank Fusion

Источник: Gordon V. Cormack, Charles L. A. Clarke, Stefan Büttcher. Reciprocal Rank Fusion outperforms Condorcet and individual Rank Learning Methods. *SIGIR 2009. PDF.*

Представим, что у нас несколько ранжировщиков и у каждого свой порядок документов. Складывать их сырые оценки нельзя: у BM25 и у косинуса совершенно разные шкалы. RRF смотрит только на место документа в списке:

$$\mathrm{RRF}(d) = \sum_{r \in R} \frac{1}{k + r(d)}, \qquad k = 60.$$

Если по-человечески, то чем выше документ в чьём-то списке, тем больше очков он получает, но никто не обнуляется. Константу $k = 60$ авторы подобрали на пилоте и дальше не трогали. Она приглушает систему, которая слишком уверенно ставит что-то на первое место: высокое место важно, но документ из середины списка всё равно получает свою долю.

На наборах TREC метод в среднем на 4–5 % MAP (средней точности по запросам) обошёл и лучшую отдельную систему, и Condorcet. CombMNZ иногда оказывался чуть выше, но разница статистически незначима. Отдельный плюс — методу не нужны ни учитель, ни обучающие пары.

Для нашего задания сливать через RRF четыре SKU нечего: мы и так смотрим все четыре. А вот сливать два ранга характеристик внутри одного SKU смысл есть. Это лексический ранг (канон и пересечение слов) и, когда он появится, ранг по эмбеддингам FRIDA. Такое слияние и будет честным экспериментом H2, если оба ранга записывать до кодовой проверки. RRF при этом не ставит `match`, он только выбирает, какую характеристику передать в `check_constraints`.

Когда появится агент, правило останется прежним: RRF живёт внутри инструмента выравнивания. Это не отдельный агент и не способ обойти проверку чисел.

6 LoRA

Источник: Edward Hu и др. LoRA: Low-Rank Adaptation of Large Language Models. *arXiv:2106.09685, 2021. arXiv.*

Полное дообучение большой модели хранит и учит все её веса. LoRA поступает иначе: замораживает их и учит только небольшую добавку низкого ранга $\Delta W = BA$, где B и A — узкие матрицы. Для GPT-3 175B авторы показали, что хватает ранга 1–2 при полной ширине 12 288. Обучаемых весов становится до десяти тысяч раз меньше, памяти под оптимизатор Adam — примерно втрое меньше. А на инференсе BA можно просто прибавить к исходной матрице W, так что лишней задержки нет. В итоге получается одна базовая модель и много маленьких адаптеров под разные задачи.

Для нашего задания LoRA не нужна. ТЗ прямо говорит, что обучать свою модель необязательно. Да и учить нечего: весов GPT-Luna у нас нет, тысяч размеченных пар «требование — вердикт» тоже. К тому же дообучать вердикт опасно, потому что модель начнёт ставить статус там, где ТЗ требует кода.

Если система будет расти, это запасной путь, но не первый. Допустим, H2 покажет, что готовая FRIDA путает «объём резервуара» и «объём заполнения». Тогда проще разметить характеристики и повесить LoRA на энкодер, чем учить модель с нуля. А адаптер для извлечения требований из прозы имеет смысл только тогда, когда ручная выборка `eval/held_out_draft.json` покажет систематический пропуск, а не случайный. Пока пропуск не измерен, LoRA просто нечему учить.

7 Что подтвердилось на практике (обновление от 6 ноября 2026)

Когда мы закрыли ручную разметку `eval/held_out_draft.json` и запустили эксперимент H2, выводы из статей выше сработали почти буквально:

1. **FRIDA + префиксы.** Включение `ai-forever/FRIDA` с префиксами `search_query:` и `search_document:` по семантическому слоту характеристики сразу закрыло перефразы, на которых падал лексический поиск («Регулирование скорости инфузии» ↔ «Тип скорости: Постоянная», «Путь введения: Внутривенный» ↔ «Внутривенное введение»), и подняло `Recall@3` на проверенной выборке с 0,833 до **1,000**.
2. **RRF с $k = 60$ плюс метрологический фильтр.** Точно как у Кормака и в гайде Raft, гибрид dense + lexical превзошёл оба канала по отдельности: точность выравнивания выросла с 0,667 (лексика) и 0,867 (чистая FRIDA) до **0,933**, а на полном корпусе открыла два стопроцентно доказанных совпадения закупок (`CT0050L` в закупках 07 и 10).
3. **Никакого LoRA и никакого реранкера.** Опасение из раздела про LoRA (вдруг готовая FRIDA перепутает номинальный и максимальный объёмы и придётся её дообучать) решилось за 0 токенов тонким фильтром метрологических квалификаторов поверх RRF (`G11` ушёл из ложного `match` в честный `unknown`).
4. **Что осталось.** Проверить retrieve-then-read по страницам длинных закупок 02 и 03 (**H5**) и каскадный вызов `gpt-5.6-luna` только на сложных `unknown` (**H7**/**H8**) при наличии живого ключа API.